Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

7. Quản lý dữ liệu — Database per Service & CQRS . . . 2
7.1. Database-per-Service — nguyên tắc Data Ownership . . . 3
7.1.1. Vấn đề: shared database phá vỡ tính độc lập của dịch vụ . . . 3
7.1.2. Database-per-service pattern . . . 4
7.1.3. Case study thực tế: Uber — Domain-Oriented Microservice Architecture (DOMA) . . . 6
7.1.4. CAP Theorem — hiểu đúng giới hạn . . . 6
7.1.5. Caching Strategies — giảm tải và độ trễ . . . 8
7.2. Chiến lược tách database từ Monolith . . . 10
7.2.1. Vấn đề: tách service dễ, tách dữ liệu khó . . . 10
7.2.2. Năm chiến lược tách database . . . 10
7.2.3. Áp dụng cho KBM . . . 11
7.3. Data Duplication vs Coupling — khi nào chấp nhận duplicate? . . . 12
7.3.1. Vấn đề: cần dữ liệu từ service khác . . . 12
7.3.2. Ba chiến lược truy cập dữ liệu xuyên service . . . 12
7.3.3. Khi nào chấp nhận duplication? . . . 14
7.4. CQRS — Command Query Responsibility Segregation . . . 15
7.4.1. Từ CQS đến CQRS . . . 15
7.4.2. Vấn đề: cùng model cho read và write không đủ . . . 16
7.4.3. CQRS pattern . . . 16
7.4.4. Khi nào cần CQRS? . . . 17
7.4.5. Ví dụ: Leaderboard trong chế độ thi — vì sao CQRS bất đồng bộ không hợp với bảng xếp hạng trực tiếp . . . 17
7.5. Event Sourcing — lưu event thay vì state . . . 19
7.5.1. Vấn đề: mất lịch sử khi chỉ lưu state . . . 19
7.5.2. Event Sourcing pattern . . . 20
7.5.3. Snapshot Pattern — giải quyết hiệu năng của event replay . . . 21
7.5.4. Projection Rebuilds — tạo lại view từ event . . . 22
7.5.5. Tiến hóa lược đồ sự kiện (Event Schema Evolution) . . . 24
7.5.6. Khi nào dùng Event Sourcing? . . . 24
7.6. Case study: KBM . . . 24
7.6.1. Hiện trạng . . . 25
7.6.2. Từ hiện trạng đến kiến trúc mục tiêu . . . 25
7.6.3. Lộ trình chuyển đổi đề xuất . . . 26
7.7. Tổng kết . . . 28
7.8. Đọc thêm . . . 29
Tài liệu tham khảo . . . 29

CHƯƠNG 7

Quản lý dữ liệu — Database per Service & CQRS

> "Duplication is far better than coupling. Each service should own its data and make independent decisions about what to store."
> — Sam Newman, *Monolith to Microservices* [4b]

MỤC TIÊU HỌC TẬP

- Hiểu nguyên tắc **database-per-service** và tại sao quyền sở hữu dữ liệu là nền tảng của microservices
- Nắm được các chiến lược tách database từ monolith — từ shared database đến database-per-service
- Phân biệt khi nào data duplication là chấp nhận được và khi nào nó là vấn đề
- Hiểu CQRS (Command Query Responsibility Segregation) và khi nào nó cần thiết
- Nắm tổng quan Event Sourcing — ưu/nhược điểm và khi nào nên áp dụng
- Phân tích bài toán truy vấn xuyên service (cross-service query) trong KBM

Bản đồ khái niệm: Ch.5 → Ch.6 → Ch.7

Ba chương cuối Phần II nối với nhau theo một mạch: **Ch.5** cho chúng ta cách các service nói chuyện bất đồng bộ qua message/event; **Ch.6** dùng chính các message đó để giữ tính nhất quán khi một nghiệp vụ trải trên nhiều service (Saga); và **Ch.7** (chương này) trả lời câu hỏi sâu hơn: nếu mỗi service *sở hữu database riêng*, thì lưu trữ, truy vấn và đồng bộ dữ liệu giữa chúng ra sao? Nói gọn: Ch.5 = "gửi tin", Ch.6 = "phối hợp giao dịch", Ch.7 = "quản lý dữ liệu của từng service". Nếu khối lượng kiến thức quá lớn, độc giả chỉ cần ghi nhớ một nguyên lý cốt lõi cho chương này: **mỗi service quản lý dữ liệu độc lập, và chỉ chia sẻ qua API hoặc event — không bao giờ chung bảng.**

7.1. Database-per-Service — nguyên tắc Data Ownership

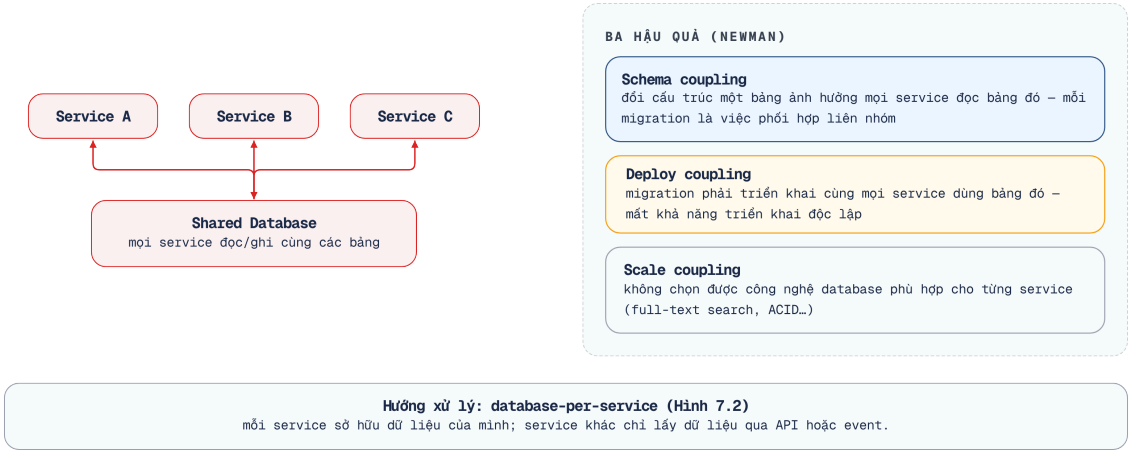
Để đạt được tính tự chủ, mỗi dịch vụ cần quản lý một không gian lưu trữ độc lập, tương tự như việc các khoa trong trường đại học tự quản lý hồ sơ sinh viên của khoa đó.

7.1.1. Vấn đề: shared database phá vỡ tính độc lập của dịch vụ

Trong monolith, toàn bộ ứng dụng truy cập một database duy nhất — đơn giản, nhất quán, và transaction ACID hoạt động tự nhiên. Khi chuyển sang kiến trúc microservices, nhiều nhóm phát triển vẫn giữ nguyên cơ sở dữ liệu dùng chung vì sự tiện lợi — mọi service vẫn đọc/ghi cùng bảng. Thoạt nhìn, đây có vẻ là giải pháp tốt: không cần thay đổi tầng dữ liệu, không cần xử lý eventual consistency.

Tuy nhiên, bài toán dữ liệu trong hệ thống phân tán bị ràng buộc bởi một giới hạn lý thuyết quan trọng — **CAP Theorem** (Consistency, Availability, Partition tolerance): trong hệ thống phân tán, khi network partition xảy ra, hệ thống chỉ có thể chọn *hoặc* consistency *hoặc* availability, không thể có cả hai [7, Ch.9]. Newman trong [4a, Ch.11] giải thích: đây là lý do microservices *thường* chấp nhận eventual consistency cho các bản sao dữ liệu liên service — nhưng mỗi thao tác vẫn phải tự chọn hành vi khi xảy ra phân mảnh mạng (chấp nhận trả lời dữ liệu có thể cũ, hay từ chối phục vụ để giữ nhất quán) — và tại sao thiết kế quản lý dữ liệu cần cân nhắc kỹ trade-off giữa consistency và availability cho *từng bài toán cụ thể*.

Shared database **phá vỡ nguyên lý cốt lõi nhất** của microservices: khả năng triển khai độc lập. Newman trong [4b, Ch.4] phân tích ba hậu quả nghiêm trọng:



Hình 7.1: Ba hậu quả của shared database trong microservices

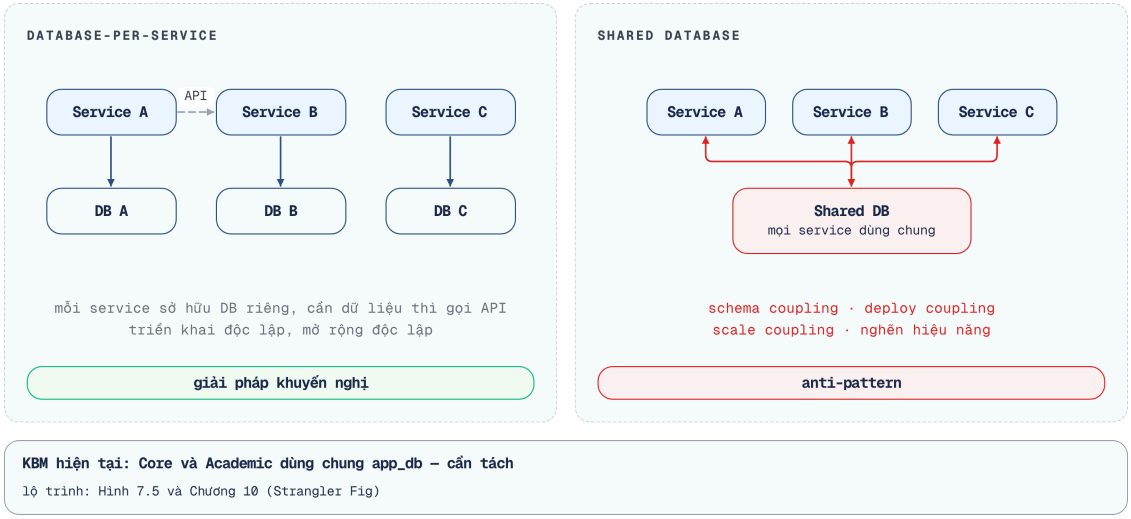
1. Schema coupling — Khi hai service đọc cùng bảng `users`, thay đổi schema (đổi tên cột, thêm constraint) ảnh hưởng cả hai. Mỗi lần cập nhật cấu trúc cơ sở dữ liệu (database migration) lại trở thành một bài toán phối hợp liên nhóm phức tạp — đây chính xác là vấn đề mà kiến trúc microservices muốn loại bỏ.

2. Deploy coupling — Database migration phải deploy cùng với tất cả service sử dụng bảng đó. Nếu Service A cần thêm cột vào `users`, Service B phải cập nhật mã nguồn để xử lý cột dữ liệu mới *trước khi* quá trình migration được thực thi — mặc dù Service B hoàn toàn không sử dụng đến trường thông tin đó.

3. Scale coupling — Không thể mở rộng database cho riêng một service: một service tải nặng làm chậm mọi service dùng chung. Cũng không chọn được công nghệ phù hợp cho từng service (Elasticsearch cho tìm kiếm, PostgreSQL cho ACID transaction).

7.1.2. Database-per-service pattern

Nguyên tắc **database-per-service** yêu cầu: mỗi service sở hữu dữ liệu riêng, và **service khác không bao giờ truy cập trực tiếp vào database đó** — dữ liệu chỉ được cung cấp ra ngoài qua API hoặc event contract do service sở hữu kiểm soát [2a, Ch.1].



Hình 7.2: *Shared Database (đọc/ghi chung) vs Database-per-Service (dữ liệu qua API)*

"Database riêng" không nhất thiết là database server riêng. Có ba cấp độ tách:

Bảng 7.1: *Ba cấp độ tách database*

Cấp độ	Mô tả	Isolation	Chi phí
Separate schema	Cùng DB server, khác schema/namespace	Thấp (vẫn có thể cross-schema query)	Thấp
Separate database	Cùng DB server, khác database	Trung bình	Thấp
Separate server	Khác DB server hoàn toàn	Cao (cho phép đa công nghệ lưu trữ (polyglot persistence))	Cao

Cấp độ nào phù hợp tùy thuộc vào yêu cầu isolation. Với KBM hiện tại, **separate schema** là bước đầu tiên hợp lý — chi phí thấp nhưng ngăn được schema coupling ngầm.

Data Should Be Owned, Not Shared

> "If a service wants to access data held by another service, it should go and ask that service for the data it needs. And the service exposing the data should decide what to expose and what to hide."
>
> — *Sam Newman, Monolith to Microservices [4b]*

Đối chiếu với nguyên tắc này, việc dùng chung cơ sở dữ liệu giữa Core và Academic trong KBM là một khoản nợ kỹ thuật chuyển tiếp cần được quản lý rõ ranh giới:

KBM shared database

KBM hiện sử dụng **shared database** (`app_db`) giữa Core Service và Academic Service (`kbm-academic`) — cả hai service đọc/ghi cùng PostgreSQL instance, có khả năng truy cập chéo bảng. Đây là hệ quả chuyển tiếp của việc Academic được tách ra từ Core (cả hai ban đầu thuộc cùng monolith). Với đội ngũ 1–2 lập trình viên chính, đây là **nợ kỹ thuật chuyển tiếp được chấp nhận có điều kiện** (chưa bắt buộc tách máy chủ vật lý ngay), nhưng nếu thiếu ranh giới sở hữu rõ ràng, thay đổi schema của Academic có thể ảnh hưởng Core và ngược lại — cản trở khả năng triển khai độc lập.

Lộ trình chuyển đổi: (1) kiểm kê bảng, xác định một chủ sở hữu ghi duy nhất cho từng bảng (dựa vào phân tích bounded context ở Ch.2) và khóa một pipeline migration duy nhất (tắt `JPA auto-DDL` cạnh tranh), (2) tách separate schema kèm database role riêng trong cùng PostgreSQL, (3) thay thế các truy vấn chéo bảng (cross-table query) bằng lời gọi API hoặc bản sao sự kiện tùy bài toán, (4) dài hạn: chỉ tách database server vật lý khi phát sinh nhu cầu mở rộng độc lập hoặc đa công nghệ lưu trữ.

7.1.3. Case study thực tế: Uber — Domain-Oriented Microservice Architecture (DOMA)

Uber (2020) công bố bài học từ việc quản lý **khoảng 2.200 microservice** — trong đó quản trị phụ thuộc và quyền sở hữu dữ liệu xuyên hàng nghìn service là những thách thức trung tâm. Adam Gluck mô tả cách Uber chuyển từ "microservices spaghetti" sang **DOMA** (Domain-Oriented Microservice Architecture):

Kiến trúc của Uber trải qua ba giai đoạn. Giai đoạn đầu (2012–2016), việc chuyển đổi từ monolith sang microservices diễn ra tự do, dẫn đến gia tăng nhanh chóng số lượng service và phức tạp hóa biểu đồ phụ thuộc.

Trong giai đoạn 2016–2018, hệ thống rơi vào trạng thái "microservices spaghetti" với hàng nghìn dịch vụ giao tiếp và truy cập dữ liệu chéo nhau, phá vỡ tính tự chủ.

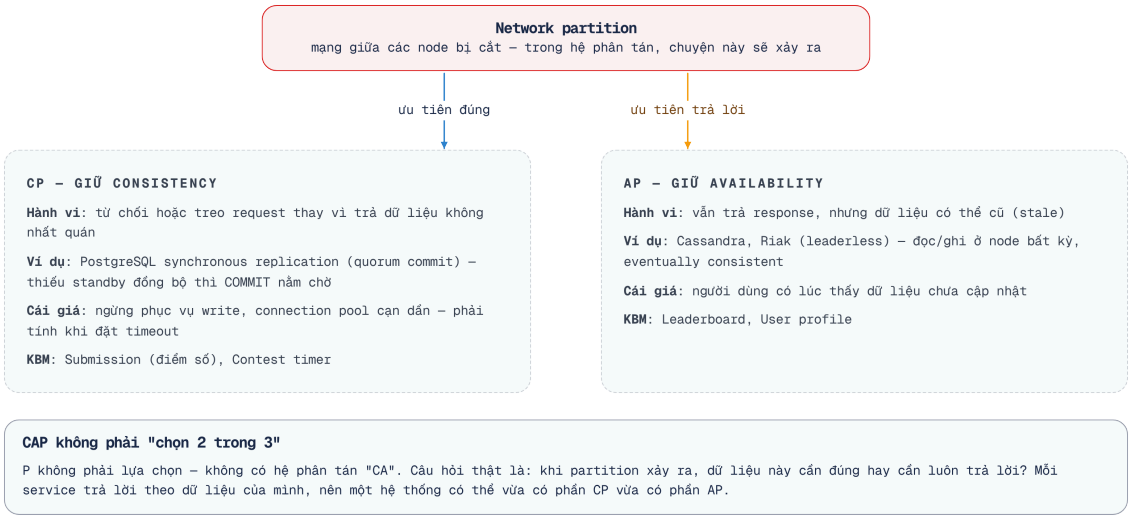
Từ năm 2018, Uber áp dụng DOMA (Domain-Oriented Microservice Architecture) bằng cách gom nhóm các dịch vụ thành những miền (domain) chức năng. Mọi giao tiếp từ bên ngoài bắt buộc đi qua một domain gateway duy nhất, qua đó giảm thiểu sự kết dính dữ liệu.

Bài học rút ra cho quản lý dữ liệu (giáo trình tổng hợp từ case study trên): (1) Database-per-service là cần thiết nhưng *chưa đủ* — cần thêm *domain-level data encapsulation*. (2) Khi hệ thống lớn, nhóm các service thành domain giúp kìm đà tăng của số phụ thuộc chéo. (3) Data duplication giữa các domain là chấp nhận được — domain gateway kiểm soát dữ liệu nào được bộc lộ ra ngoài.

Nguồn: Adam Gluck, "Introducing Domain-Oriented Microservice Architecture," Uber Engineering Blog, July 2020 (eng.uber.com/microservice-architecture/). Xem thêm: Uber Technology Day 2019 talks.

7.1.4. CAP Theorem — hiểu đúng giới hạn

CAP Theorem được đề cập ngắn gọn ở trên, nhưng hiểu *đúng* CAP rất quan trọng khi thiết kế quản lý dữ liệu. Kleppmann trong [7, Ch.9] phân tích kỹ: CAP không phải "chọn 2 trong 3" như thường hiểu sai — mà là: **khi network partition xảy ra** (và nó *sẽ* xảy ra trong hệ thống phân tán), hệ thống phải chọn giữa consistency và availability.



Hình 7.3: CAP Theorem — CP (chọn consistency) vs AP (chọn availability) khi partition

CP Systems (Consistency + Partition tolerance): khi partition xảy ra, hệ thống ưu tiên consistency — từ chối request thay vì trả data không nhất quán. Ví dụ: PostgreSQL với synchronous replication (quorum commit) đặt consistency lên trên — khi không đủ standby đồng bộ, primary **giữ COMMIT ở trạng thái chờ** thay vì xác nhận một ghi có thể mất. Hệ quả vận hành: dịch vụ ngừng phục vụ write (và connection pool cạn dần) chứ không trả về lỗi nhanh — cần tính đến khi đặt timeout.

AP Systems (Availability + Partition tolerance): khi partition xảy ra, hệ thống ưu tiên availability — vẫn trả response nhưng data có thể stale. Ví dụ: Cassandra, Riak (kiến trúc Leaderless) cho phép đọc/ghi ở bất kỳ node nào — data *eventually* consistent.

Bảng 7.2: CP vs AP cho từng luồng dữ liệu trong KBM

Luồng dữ liệu / Nghiệp vụ	Loại dữ liệu	Nên CP hay AP?	Lý do
Kết quả chấm / điểm	Transactional	**CP**	Điểm phải đúng: thà tạm từ chối ghi còn hơn ghi điểm sai, dẫn đến khiếu nại. (Việc *tiếp nhận* bài nộp vẫn ưu tiên sẵn sàng — Ch.6.)
Leaderboard	Derived/read	**AP**	Chấp nhận tính nhất quán cuối. Người dùng thà thấy bảng xếp hạng cập nhật chậm vài giây còn hơn là không thể truy cập dịch vụ.
User profile	Reference	**AP**	Tên sinh viên rất hiếm khi thay đổi, do đó việc tồn tại dữ liệu cũ (stale data) trong vài phút không gây ra tác động nghiêm trọng

Luồng dữ liệu / Nghiệp vụ	Loại dữ liệu	Nên CP hay AP?	Lý do
Lượt nộp giới hạn trong kỳ thi	Transactional	**CP**	Số lượt nộp tối đa của thí sinh phải nhất quán chặt để đảm bảo công bằng; không cho phép ghi vượt hạn mức ở hai nhánh phân mảnh

Một lưu ý quan trọng: CAP Theorem *chỉ* phát biểu về sự đánh đổi giữa Consistency và Availability khi network partition thực sự xảy ra. Trong điều kiện bình thường (không có partition), hệ thống không bị buộc phải từ chối phục vụ như khi đứt mạng, nhưng theo định lý mở rộng **PACELC** (Daniel Abadi), hệ thống có nhân bản dữ liệu vẫn phải đánh đổi giữa **độ trễ** (chữ *L*) và **tính nhất quán** (chữ *C*) — tức chờ xác nhận đồng bộ từ các bản sao hay trả lời ngay trên node cục bộ. Hiểu đúng cả hai vế giúp tránh thiết kế quá phòng thủ — không mặc định chấp nhận eventual consistency ở những nơi một cơ sở dữ liệu chính hoặc quorum nội bộ độ trễ thấp đã đáp ứng tốt.

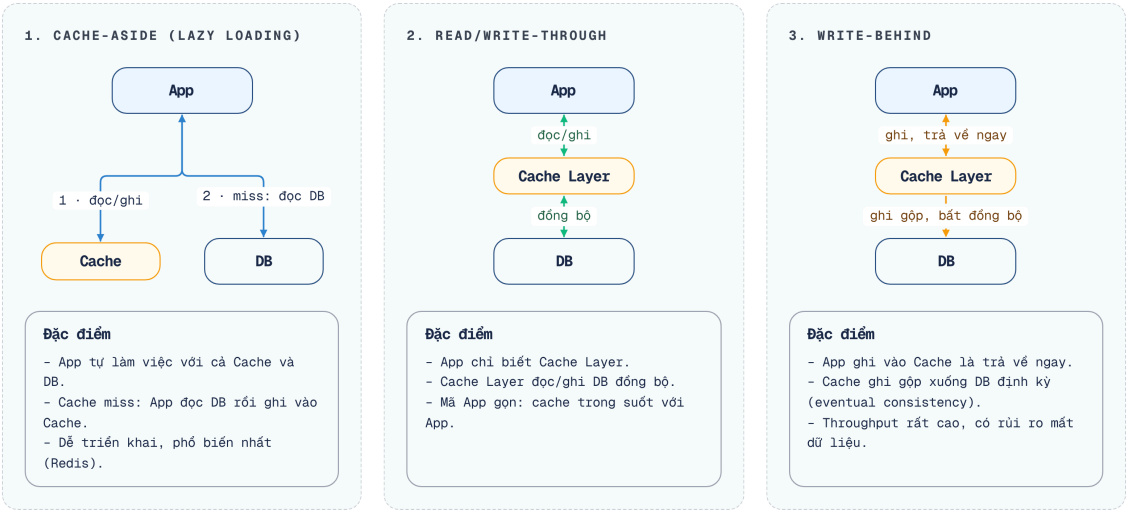
Per-Service CAP Decision

Không phải toàn bộ hệ thống "là CP" hay "là AP". Mỗi service, thậm chí mỗi *bài toán*, chọn trade-off riêng. Kleppmann trong [7, Ch.9] gọi đây là "real-world application of CAP": phân tích *từng luồng dữ liệu* — dữ liệu nào cần strong consistency (mọi lần đọc thấy bản ghi mới nhất), dữ liệu nào chấp nhận eventual consistency. Lưu ý phân biệt: chữ C của CAP nói về tính nhất quán giữa các bản sao khi đọc/ghi, khác với chữ C trong ACID (transaction bảo toàn ràng buộc nghiệp vụ) — hai khái niệm trùng tên nhưng không trùng nghĩa.

7.1.5. Caching Strategies — giảm tải và độ trễ

Khi cơ sở dữ liệu đã được phân tách, các truy vấn liên dịch vụ (cross-service query) sẽ có độ trễ cao hơn do phải thực hiện qua mạng (network call). Sử dụng bộ nhớ đệm (caching) là cách phổ biến để giảm độ trễ này, nhưng đòi hỏi chiến lược phù hợp để tránh tình trạng dữ liệu lỗi thời (stale data).

Các pattern chính (Newman [4a, Ch.11]):



Hình 7.4: Các caching pattern — Cache-Aside, Read-Through/Write-Through, Write-Behind

Mỗi pattern có ưu nhược điểm riêng và phù hợp với các luồng dữ liệu khác nhau. Việc lựa chọn phụ thuộc vào yêu cầu của từng dịch vụ.

Bảng 7.3: So sánh ba caching pattern

Pattern	Ưu điểm	Nhược điểm	Có thể áp dụng cho KBM
Cache-Aside	Đơn giản, app kiểm soát	Khi không tìm thấy dữ liệu trong bộ đệm (Cache miss), hệ thống phải thực hiện hai luồng truy vấn (tới cache và database) làm tăng độ trễ	Question list, user profile
Read-Through	Trong suốt (transparent) đối với ứng dụng	Cache layer phức tạp hơn	—
Write-Behind	Write nhanh (async)	Nguy cơ mất dữ liệu nếu cache gặp sự cố	(ví dụ chung) bộ đếm lượt xem, ghi dồn theo lô

Cache Invalidation — "There are only two hard things in Computer Science: cache invalidation and naming things." (Phil Karlton).

Chiến lược cache invalidation xoay quanh ba phương pháp chính:

- **TTL (Time-to-Live)**: Dữ liệu tự hết hạn sau một khoảng thời gian định trước. Đơn giản nhất, phù hợp với dữ liệu tham chiếu ít thay đổi như danh sách câu hỏi.
- **Event-driven invalidation**: Cache chỉ bị xóa khi hệ thống gốc phát sự kiện thay đổi. Hiệu quả hơn cho dữ liệu biến động liên tục như điểm số.
- **Versioned keys**: Gắn số phiên bản vào khóa cache. Thường dùng để dọn hàng loạt khi cấu trúc dữ liệu thay đổi lớn.

Trong KBM, bên cạnh thiết kế rate limiting theo token bucket ở Gateway (Ch.8), kiến trúc mục tiêu mở rộng vai trò của Redis sang caching và bảng xếp hạng trực tiếp (mục sau) — tận dụng TTL tự động và cơ chế pub/sub cho các sự kiện làm mới bộ đệm (invalidation event).

7.2. Chiến lược tách database từ Monolith

Chia nhỏ cơ sở dữ liệu thường gặp khó khăn tương tự như khi trường đại học tách phòng ban: tách nhân sự (code) thì nhanh chóng, nhưng bóc tách hồ sơ giấy tờ (dữ liệu) lại phức tạp hơn nhiều.

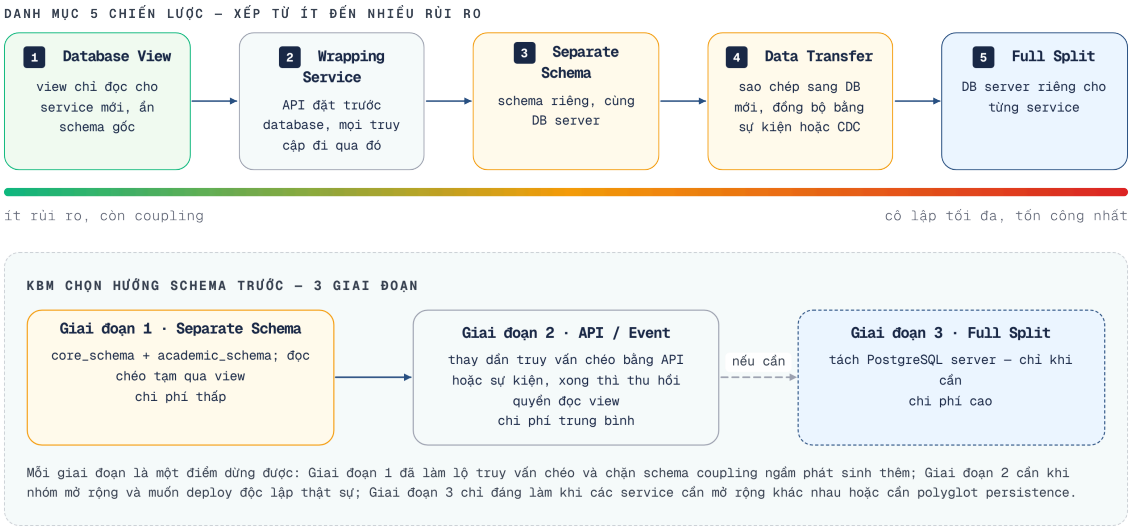
7.2.1. Vấn đề: tách service dễ, tách dữ liệu khó

Nhiều nhóm phát triển tách microservices ở tầng application (deploy riêng, repo riêng) nhưng vẫn **trỏ vào cùng database**. Newman trong [4b] gọi đây là "half-baked migration" — nửa vời. Service đã "tách" nhưng dữ liệu vẫn bị gắn chặt — lợi ích triển khai độc lập gần như bằng 0.

Tách database là **một trong những bước khó nhất** trong quá trình chuyển đổi từ monolith sang microservices. Kleppmann trong [7, Ch.5–6] phân tích: khi dữ liệu nằm ở nhiều nơi, mọi vấn đề của hệ thống phân tán xuất hiện — replication lag, partition tolerance, consistency trade-off. Đây là chi phí không thể tránh.

7.2.2. Năm chiến lược tách database

Newman đề xuất năm chiến lược, sắp xếp từ ít rủi ro đến nhiều rủi ro [4b, Ch.4]:



Hình 7.5: *Năm chiến lược tách database — danh mục tổng quát xếp từ ít đến nhiều rủi ro (nhãn "lộ trình" trong hình mô tả trình tự của danh mục này; KBM chỉ chọn lọc 3 giai đoạn phù hợp — xem bảng dưới)*

1. Database View — Tạo view read-only cho service mới, ẩn schema gốc. Ưu điểm: không thay đổi dữ liệu, service cũ không bị ảnh hưởng. Nhược điểm: chỉ read-only, vẫn coupled vào schema.

2. Database Wrapping Service — Đặt một API đơn giản phía trước database, mọi truy cập đi qua API này. Ưu điểm: bắt đầu tách coupling mà không di chuyển dữ liệu. Nhược điểm: làm phát sinh thêm một dịch vụ cần được bảo trì (maintain).

3. Separate Schema — Tách bảng vào schema/namespace riêng trong cùng database. Ưu điểm: sự cô lập (isolation) rõ ràng, không yêu cầu thiết lập hạ tầng (infrastructure) mới. Nhược điểm: cùng DB server → vẫn dùng chung tài nguyên (shared resource).

4. Data Transfer — Sao chép dữ liệu sang database mới, giữ đồng bộ bằng event hoặc CDC (Change Data Capture). Ưu điểm: service mới có dữ liệu riêng. Nhược điểm: cần cơ chế đồng bộ, eventual consistency.

5. Full Database Split — Mỗi service có database server hoàn toàn riêng. Ưu điểm: isolation tối đa, cho phép đa công nghệ lưu trữ (polyglot persistence). Nhược điểm: Đòi hỏi nhiều công sức triển khai và phải xử lý phức tạp các truy cập dữ liệu liên dịch vụ (cross-service data access).

7.2.3. Áp dụng cho KBM

Danh mục năm chiến lược cho biết *có những bước nào* câu hỏi còn lại là *bước nào đi trước*. Với hai service đang dùng chung `app_db`, có hai hướng hợp lý. Chúng dùng cùng các chiến lược và đi đến cùng một đích — chỉ khác ở chỗ dựng ranh giới dữ liệu trước hay gỡ truy vấn chéo trước.

- **Hướng A — schema trước**: tách `app_db` thành `core_schema` và `academic_schema` ngay trong cùng PostgreSQL, mỗi service một database role. Những truy vấn chéo còn lại tạm đọc qua view được cấp quyền rõ ràng, nên Academic chưa phải đổi code. Từ đây mọi truy cập chéo phải gọi đích danh schema, nên đếm và theo dõi được; lần lượt thay chúng bằng API, rồi mới thu hồi quyền đọc chéo. Được: chi phí thấp, ranh giới thấy được ngay, và biết chính xác cần viết những API nào. Trả giá: trong giai đoạn đầu ranh giới còn "mềm" — vẫn dựa vào quyền đọc tạm thời và kỷ luật của nhóm.
- **Hướng B — API trước**: đây là thứ tự tổng quát trình bày ở Ch.10 (bảng năm chiến lược theo góc độ chuyển đổi): thay truy vấn chéo bằng API (hoặc view) trước; khi không còn truy vấn chéo nào, việc tách schema gần như không còn rủi ro. Được: bước tách schema an toàn, mỗi bước đều rollback được. Trả giá: lâu mới có ranh giới ở tầng dữ liệu, và phải đoán trước truy vấn chéo nào cần API khi chưa có số liệu.

Hướng B an toàn hơn khi nhiều service và nhiều nhóm cùng chạm vào một database: không ai phải chịu một đợt đổi schema lớn khi truy vấn chéo còn nằm rải rác. Hướng A hợp khi chỉ có vài service, một DB server và một nhóm nhỏ: dựng ranh giới rẻ, và số liệu truy vấn chéo thu được giúp tránh viết API thừa.

KBM chọn **hướng A**. Chỉ Core và Academic dùng chung `app_db`, trên cùng một PostgreSQL, với đội phát triển một đến hai người. Rủi ro lớn nhất hiện nay là schema coupling ngầm — chưa ai biết chắc Academic đang đọc những bảng nào của Core — nên bước có giá trị nhất là một ranh giới rẻ làm lộ ra các truy vấn đó:

__Bảng 7.4:__ Lộ trình tách database cho KBM theo hướng schema trước (3 giai đoạn chọn lọc từ danh mục 5 chiến lược)

Giai đoạn	Chiến lược	Mô tả	Chi phí
1	**Separate Schema**	Kiểm kê chủ sở hữu bảng, tắt auto-DDL cạnh tranh; tách `app_db` thành `core_schema` và `academic_schema` với role riêng, các truy vấn chéo tạm đọc qua view	Thấp
2	**API / Event theo bài toán**	Thay dần truy vấn chéo bằng API hoặc bản sao sự kiện tùy luồng đọc/ghi; hết truy vấn chéo thì thu hồi quyền đọc view	Trung bình
3	**Full Split** (nếu cần)	Tách PostgreSQL server vật lý khi phát sinh nhu cầu mở rộng độc lập	Cao

Khi nào đủ?

Không phải hệ thống nào cũng cần đến Giai đoạn 3. Với KBM, Giai đoạn 1 (separate schema) đủ để làm lộ toàn bộ truy vấn chéo và chặn schema coupling ngầm phát sinh thêm. Giai đoạn 2 (API / Event theo bài toán) cần khi nhóm mở rộng và cần deploy độc lập thực sự. Giai đoạn 3 chỉ cần khi có yêu cầu mở rộng khác nhau hoặc polyglot persistence.

Chi tiết lộ trình chuyển đổi, Outbox Pattern (reliable messaging khi tách DB), và thứ tự triển khai → xem Ch.10.

7.3. Data Duplication vs Coupling — khi nào chấp nhận duplicate?

Trong kiến trúc microservices, việc nhân bản dữ liệu thường được áp dụng để đảm bảo tính độc lập. Cách làm này tương tự như việc phòng thư viện lưu bản sao thông tin thẻ sinh viên từ phòng đào tạo để tra cứu nhanh mà không cần đối chiếu chéo liên tục.

7.3.1. Vấn đề: cần dữ liệu từ service khác

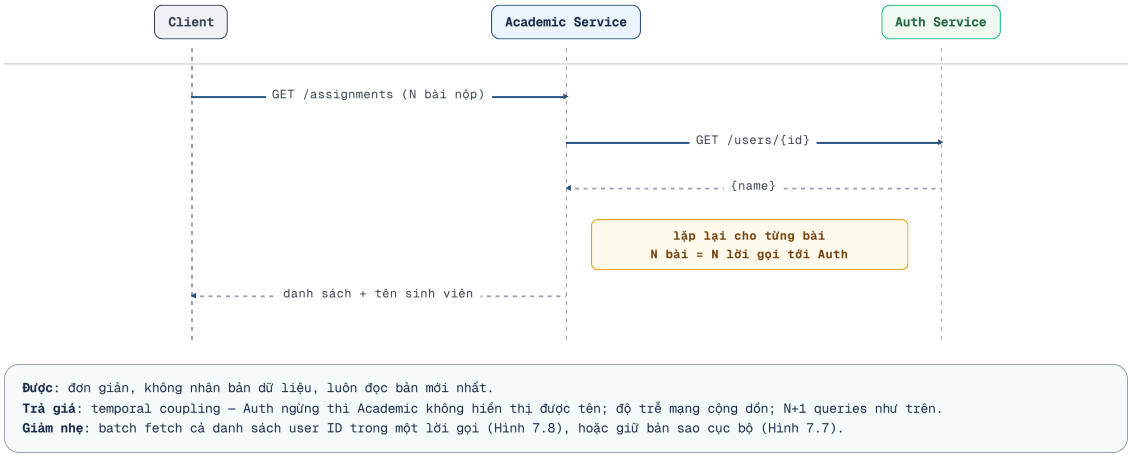
Sau khi tách database, vấn đề tiếp theo lập tức xuất hiện: **service A cần dữ liệu mà service B sở hữu**. Ví dụ trong KBM: Academic Service cần hiển thị tên sinh viên — nhưng `users` thuộc về Auth Service. Gọi Auth API mỗi lần cần tên? Lưu bản sao tên sinh viên tại Academic?

Mitra trong [3, Ch.5] phân tích ba giải pháp, mỗi cách có trade-off riêng:

7.3.2. Ba chiến lược truy cập dữ liệu xuyên service

1. API Call (Runtime dependency)

Cách đơn giản nhất là gọi API của service sở hữu dữ liệu mỗi khi cần (Hình 7.6).

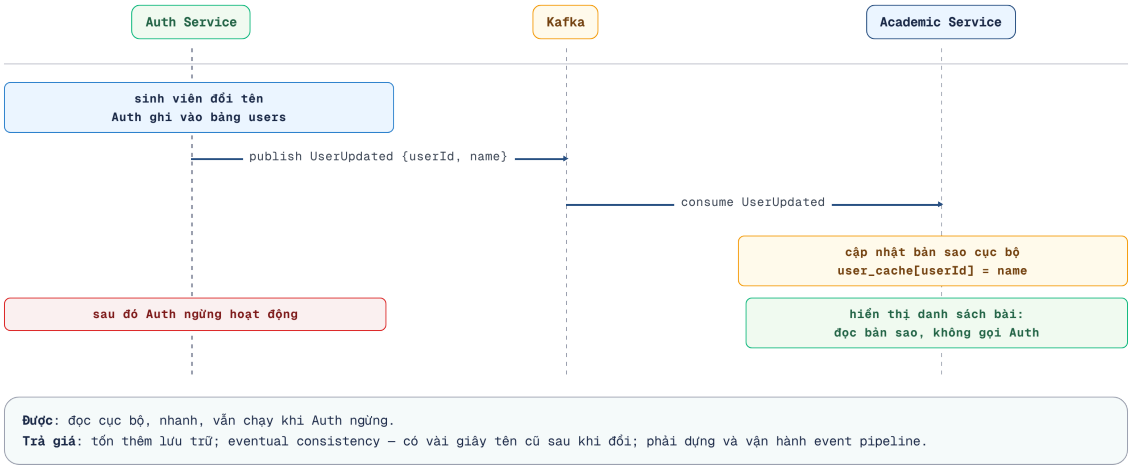


Hình 7.6: *Truy cập dữ liệu xuyên service bằng API Call*

API Call được chọn vì đơn giản khi triển khai, không nhân bản dữ liệu và luôn đọc được thông tin mới nhất. Cái giá phải trả là temporal coupling: dịch vụ gọi bị gián đoạn theo dịch vụ đích (Auth ngừng hoạt động thì Academic không hiển thị được tên người dùng), độ trễ mạng cộng dồn, và luồng đọc dễ rơi vào bài toán N+1 truy vấn (N+1 query).

2. Data Duplication (Local copy)

Cách thứ hai là giữ một bản sao cục bộ, được cập nhật qua event pipeline từ service sở hữu (Hình 7.7).

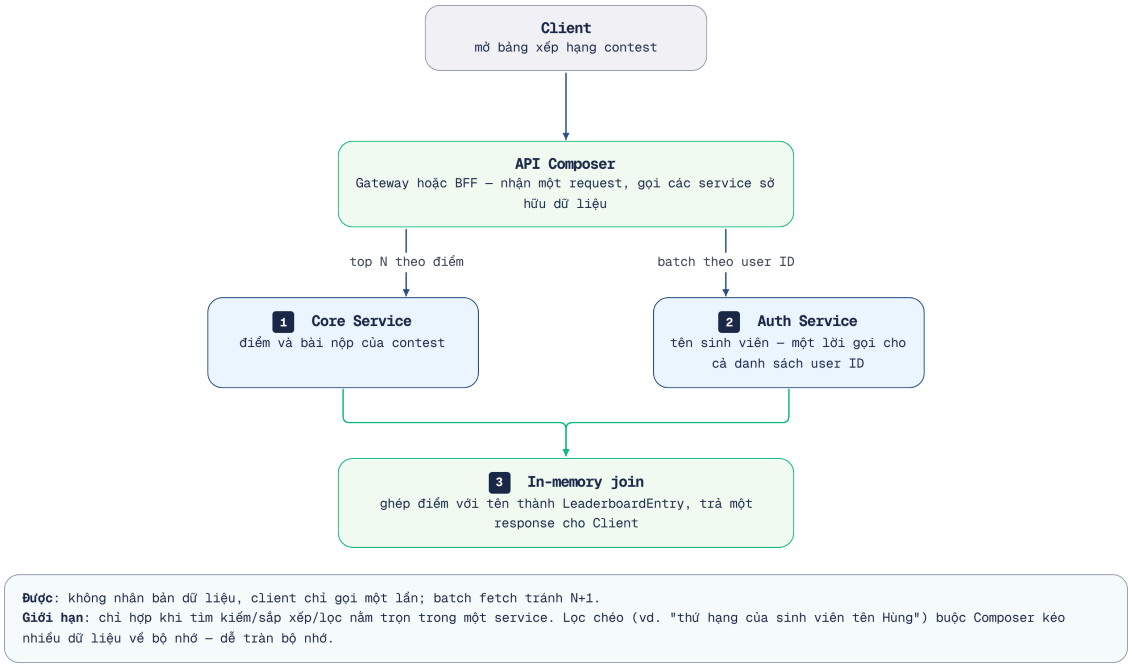


Hình 7.7: *Truy cập dữ liệu xuyên service bằng Data Duplication qua Event Pipeline*

Ngược lại, Data Duplication khắc phục điểm yếu của API Call bằng cách loại bỏ sự phụ thuộc thời gian thực và cho phép truy vấn dữ liệu cục bộ với tốc độ cao ngay cả khi các dịch vụ khác đang gặp sự cố (downtime). Đổi lại, hệ thống phải chấp nhận tốn thêm dung lượng lưu trữ, đối mặt với tính nhất quán cuối (eventual consistency) và tình trạng dữ liệu lỗi thời (stale data) tạm thời, và tốn công sức xây dựng các luồng đồng bộ sự kiện (event pipeline) phức tạp.

3. API Composition / Data Aggregation

Dùng khi cần ghép dữ liệu từ nhiều service. Một service (hoặc API Gateway) gọi nhiều service rồi tổng hợp. Richardson trong [2a, Ch.7] gọi đây là **API Composition pattern**:



Hình 7.8: API Composition pattern — Gateway hoặc BFF gọi nhiều service rồi tổng hợp

Ví dụ: hiển thị bảng xếp hạng kỳ thi cần dữ liệu từ Core (bài nộp) và Auth (tên người dùng). API Composer gọi cả hai service, sau đó **in-memory join** — thực hiện truy vấn theo lô (batch fetch) danh sách user ID để tránh lỗi N+1 truy vấn, sau đó tổng hợp dữ liệu trên bộ nhớ (in-memory aggregation) để tạo ra `LeaderboardEntry`.

⚠ Lỗi N+1 truy vấn và tràn bộ nhớ (Out-of-Memory)

Phương pháp In-memory join hoạt động rất hiệu quả nếu chỉ lấy dữ liệu hiển thị. Tuy nhiên, nếu phát sinh yêu cầu **tìm kiếm/lọc chéo** (ví dụ: "Tìm thứ hạng của sinh viên tên Hùng"), API Composer sẽ phải truy vấn toàn bộ dữ liệu từ Auth (tất cả user tên Hùng) rồi chuyển cho Core để lọc, có thể gây tràn bộ nhớ (Out-of-Memory). API Composition chỉ phù hợp khi tiêu chí Search/Sort/Filter nằm trọn trong một Service.

7.3.3. Khi nào chấp nhận duplication?

Newman trong [4b] đưa ra nguyên tắc rõ ràng: **"Duplication is far better than coupling."** Tuy nhiên, điều này không đồng nghĩa với việc cho phép nhân bản mọi loại dữ liệu một cách tùy tiện. Quy tắc:

Bảng 7.5: Khi nào chấp nhận nhân bản dữ liệu (data duplication)

Loại dữ liệu	Nên nhân bản?	Lý do
Reference data (tên user, tên khóa học)	✓ Có	Ít thay đổi, cần hiển thị ở nhiều nơi
Transactional data (điểm số, trạng thái submission)	× Không (Master-Master)	Không nhân bản 2 chiều. Sao chép 1 chiều từ Write Model sang Read Model thì bắt buộc

Loại dữ liệu	Nên nhân bản?	Lý do
Configuration (loại database, danh mục)	√ Có	Gần như tĩnh, cache tốt
Audit data (lịch sử thay đổi)	× Không (đa nguồn ghi)	Chỉ một nơi được ghi; bản sao chỉ-đọc đưa về Data Warehouse qua ETL/Data Streaming (một chiều) là hợp lệ

Nguyên tắc quan trọng nhất ở đây là phải luôn biết rõ ai đang là người làm chủ nguồn dữ liệu được sao chép.

Single Source of Truth

Nhân bản dữ liệu (duplication) chỉ chấp nhận được khi tồn tại một **single source of truth** (nguồn sự thật duy nhất) rõ ràng: Service A sở hữu dữ liệu, Service B giữ bản sao. Khi dữ liệu thay đổi, thay đổi chảy theo chiều A → B (qua event), không bao giờ theo chiều ngược lại. Nếu không thể xác định rõ ràng quyền sở hữu đối với một tập dữ liệu, đó là dấu hiệu cho thấy ranh giới ngữ cảnh (bounded context) được thiết kế chưa tốt. Trong trường hợp này, kiến trúc sư cần xem xét lại quá trình phân tích ở Chương 2.

Hiện KBM vẫn dựa vào chiến lược thứ nhất (API call) cho các truy vấn xuyên service:

Truy vấn xuyên service trong KBM

KBM hiện dùng **Feign call** để truy vấn dữ liệu xuyên service — ví dụ: Academic Service gọi Core Service để lấy danh sách câu hỏi. Đây là API call pattern (chiến lược 1). Với quy mô học thuật vừa phải, cách này chấp nhận được. Tuy nhiên, trong chế độ thi hoặc trên các bảng điều khiển (dashboard) chứa nhiều thông tin, mỗi lần tải trang có thể kích hoạt (trigger) hàng loạt các lệnh gọi Feign liên tiếp, dẫn đến độ trễ của hệ thống tăng lên đáng kể.

Lộ trình chuyển đổi: (1) xác định dữ liệu nào Academic cần *hiển thị* từ Core (chủ yếu reference data: tên câu hỏi, tên kỳ thi), (2) nhân bản dữ liệu tham chiếu (reference data) thông qua các sự kiện Kafka (`UserUpdated`, `QuestionUpdated`), (3) giữ Feign call cho các truy vấn giao dịch (transactional query) (kiểm tra điểm real-time).

7.4. CQRS — Command Query Responsibility Segregation

Hệ thống phân tán có thể tách biệt thao tác ghi và đọc dữ liệu để tối ưu hiệu suất, tương tự như việc trường học tách riêng cổng nộp hồ sơ nhập học (ghi) và bảng tin xem kết quả (đọc).

7.4.1. Từ CQS đến CQRS

Trước khi nói về CQRS, cần hiểu nguồn gốc. Bertrand Meyer đề xuất **CQS** (Command Query Separation) như một nguyên tắc thiết kế *ở cấp độ method*: mỗi method hoặc trả về dữ liệu (query, không thay đổi state) hoặc thay đổi state (command, không trả về dữ liệu) — nhưng không làm cả hai. Rocha trong [5, §4.5.1] phân biệt rõ: **CQS là nguyên tắc ở cấp single-service/single-class**, còn **CQRS mở rộng nguyên tắc này lên cấp kiến trúc hệ thống** — tách hẳn *model* (thậm chí *database*) cho read và write. CQRS không bắt buộc phải có CQS, nhưng tinh thần tương tự: tách biệt trách nhiệm đọc và ghi.

7.4.2. Vấn đề: cùng model cho read và write không đủ

Trong monolith, cùng một entity `Submission` phục vụ cả hai mục đích:

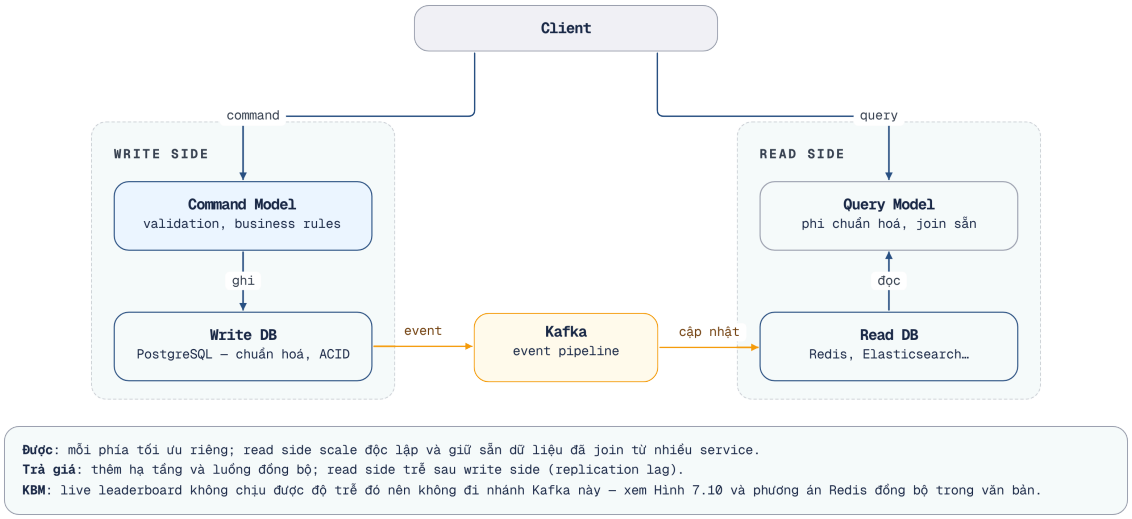
- **Write**: nộp bài (INSERT), cập nhật kết quả (UPDATE)
- **Read**: hiển thị lịch sử (SELECT nhiều cột, JOIN nhiều bảng, pagination, filtering)

Yêu cầu của các thao tác đọc và ghi là hoàn toàn khác biệt. Quá trình ghi đòi hỏi tính nhất quán (consistency), khả năng kiểm tra tính hợp lệ (validation) và phải tuân thủ chặt chẽ các quy tắc nghiệp vụ (business rule). Trong khi đó, quá trình đọc lại ưu tiên hiệu năng (performance), thường sử dụng cấu trúc dữ liệu phi chuẩn hóa (denormalized data) để hỗ trợ các truy vấn linh hoạt. Khi hệ thống phức tạp, dùng chung một mô hình dữ liệu duy nhất sẽ buộc phải dung hòa cả hai yêu cầu này, khiến không thao tác nào đạt mức tối ưu [2a, Ch.7].

Richardson trong [2a, Ch.7] mô tả vấn đề này đặc biệt nghiêm trọng khi cần **truy vấn xuyên service (cross-service query)**: hiển thị bảng xếp hạng kỳ thi (dữ liệu từ Core + Auth) bằng một truy vấn duy nhất — *không thể* khi dữ liệu nằm ở nhiều database.

7.4.3. CQRS pattern

CQRS (Command Query Responsibility Segregation) tách model thành hai phần riêng biệt [2a, Ch.7]:



Hình 7.9: CQRS pattern — tách command model (write) và query model (read)

Command side (write):

- Mô hình dữ liệu chuẩn hoá (3NF)
- Kiểm tra dữ liệu chặt, áp business rule
- Database: PostgreSQL (ACID transaction)

Query side (read):

- View phi chuẩn hoá, join sẵn
- Tối ưu cho từng kiểu truy vấn
- Database: có thể khác công nghệ lưu trữ (Elasticsearch cho tìm kiếm, Redis cho bảng xếp hạng, PostgreSQL materialized view cho báo cáo)

7.4.4. Khi nào cần CQRS?

CQRS làm tăng độ phức tạp đáng kể — **không sử dụng khi không thực sự cần thiết** [5, §4.5]:

Bảng 7.6: Khi nào cần CQRS

Kịch bản	Cần CQRS?	Lý do
Tỉ lệ đọc/ghi cân bằng, cùng model	× Không	CRUD đơn giản là đủ
Read phức tạp (cross-service join)	√ Có	Query model denormalized giải quyết join
Read volume >> Write volume	√ Có	Cho phép mở rộng quy mô mô hình đọc độc lập
UI cần real-time view (leaderboard)	√ Có	Pre-computed view nhanh hơn
Nhóm phát triển quy mô nhỏ, dự án ở giai đoạn thử nghiệm (MVP)	× Không	Chi phí đánh đổi cho sự phức tạp lớn hơn nhiều so với lợi ích thu được.

7.4.5. Ví dụ: Leaderboard trong chế độ thi — vì sao CQRS bất đồng bộ không hợp với bảng xếp hạng trực tiếp

Trong KBM, bảng xếp hạng trong chế độ thi cần dữ liệu từ Auth và Core — SQL JOIN truyền thống không dùng được khi database đã tách.

Cách tiếp cận đơn giản nhất là áp dụng CQRS qua Kafka: write side phát `ScoreUpdatedEvent`, read side tiêu thụ và cập nhật `LeaderboardEntry` phi chuẩn hóa. Tuy nhiên, cách này **không phù hợp về mặt trải nghiệm người dùng (UX)** cho kỳ thi trực tiếp (live contest). Eventual consistency tạo ra replication lag. Hơn nữa, việc áp dụng "Optimistic UI" (tự đoán thứ hạng) là **không khả thi** vì thứ hạng của thí sinh phụ thuộc vào điểm của nhiều thí sinh khác đang nộp bài cùng lúc.

Giải pháp kiến trúc đề xuất: Sử dụng **Synchronous Materialized View** qua Redis kết hợp với **WebSocket/SSE** thay vì CQRS bất đồng bộ (qua Kafka). Trong kiến trúc mục tiêu của KBM, Redis Sorted Set đóng vai trò read model trong bộ nhớ do chính `kbm-core` (hoặc `kbm-exam` khi tách phân hệ thi ở §2.4) sở hữu; ngay trên đường ghi chính, dịch vụ cập nhật điểm số vào Redis bằng lệnh `ZADD` với kỹ thuật **Composite Score** (ví dụ: `score = (real_score * 10^13) - submit_timestamp_ms`, trong đó `submit_timestamp_ms` là mốc thời gian Unix tính bằng *mili-giây*). (Nếu ở một hệ thống quy mô lớn tách hẳn một dịch vụ đọc bảng xếp hạng độc lập, dịch vụ chấm điểm sẽ gọi **gRPC/REST API đồng bộ** sang dịch vụ đọc đó thay vì chạm trực tiếp vào Redis của nó.) Composite Score giải quyết bài toán "bằng điểm" (tie-breaker), tránh việc Redis tự động xếp hạng theo bảng chữ cái của `userId`; còn tên hiển thị của sinh viên (`userId → fullName`) được ghép từ bản sao tham chiếu cục bộ/Redis hash được đồng bộ qua sự kiện `UserUpdated` từ `Auth Service`. Redis cung cấp truy vấn thứ hạng cực nhanh qua `ZREVRANK` (chính xác, `O(log N)`, in-memory), loại bỏ replication lag trên đường chính (fast path) cho người nộp bài. Tuy nhiên, để những người xem bảng xếp hạng khác thấy được sự thay đổi ngay lập tức, hệ thống vẫn cần tích hợp cơ chế đẩy từ máy chủ xuống trình duyệt (như WebSocket hoặc Server-Sent Events) để phát thông báo thay đổi thứ hạng ra toàn bộ client.

Lưu ý về giới hạn số học: Redis lưu score của Sorted Set bằng số thực dấu phẩy động 64-bit, nên chỉ biểu diễn chính xác các số nguyên trong khoảng `±2^53`, xấp xỉ `9,007 × 10^15`. Công thức trên vì vậy đòi hỏi một invariant: `real_score` là số nguyên đã chuẩn hóa theo thang 0–100. Khi đó giá trị composite tối đa khoảng `10^15`, còn cách trần an toàn khoảng chín lần. Nếu thang điểm được nới rộng vượt quá 900, phần mili-giây bắt đầu bị làm tròn và tie-breaker mất tác dụng — hai bài nộp cách nhau 1 ms có thể nhận cùng một score. Ngoài ra, hệ số `10^13` chỉ lớn hơn mốc thời gian Unix

mili-giây cho đến năm 2286, và leaderboard cần quy ước rõ giữ *thời điểm sớm nhất* khi một người dùng đạt lại cùng mức điểm.

⚠ Dual-Write (Ghi kép đồng bộ)

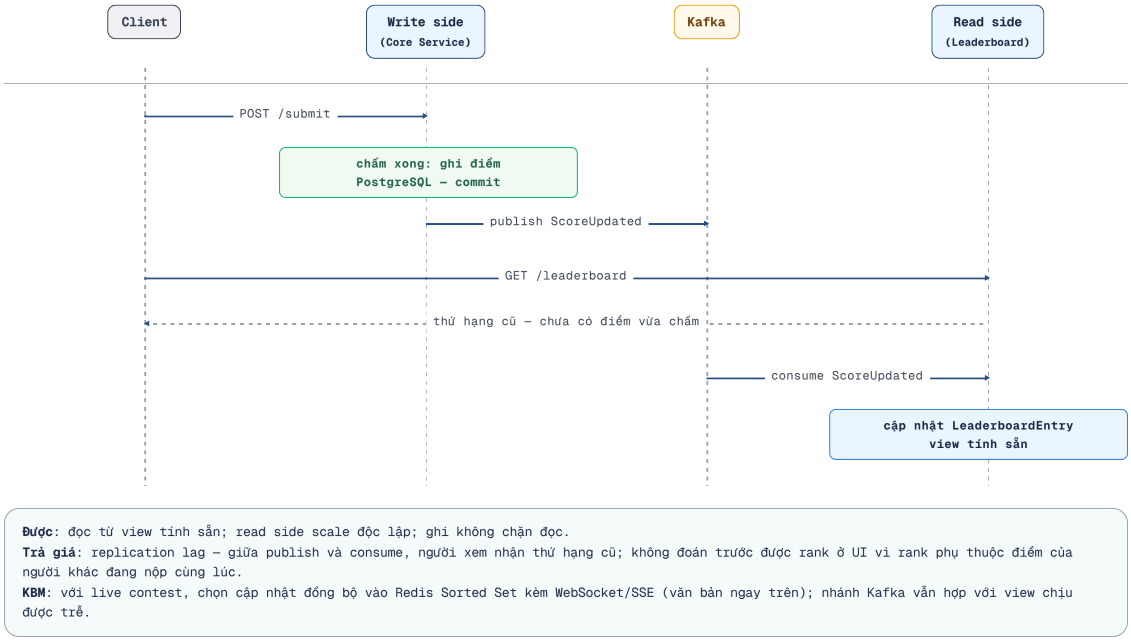
Khi thiết kế Leaderboard cập nhật đồng bộ sang kho lưu trữ thứ hai (Redis nội bộ của `kbm-core`/`kbm-exam`, hoặc gọi đồng bộ sang một dịch vụ đọc tách rời — trong trường hợp tách dịch vụ riêng, tuyệt đối **không cho phép** dịch vụ ghi chạm trực tiếp vào Redis của dịch vụ đọc để giữ Data Encapsulation), thao tác vừa commit PostgreSQL vừa cập nhật Redis luôn tiềm ẩn rủi ro **Partial Failure** (lỗi mạng hoặc tiến trình sập giữa chừng gây lệch dữ liệu).

Để khép lại rủi ro này, đường bền vững phải là **mặc định** chứ không phải phương án dự phòng: mọi thay đổi điểm được ghi vào **Transactional Outbox (Ch.10) trong cùng local transaction** với PostgreSQL, kèm một `eventId` ổn định. Nhờ đó không tồn tại cửa sổ mất dữ liệu nào, kể cả khi dịch vụ ghi sập ngay sau commit.

Bước cập nhật đồng bộ vào Redis khi đó chỉ là *fast path* để xóa độ trễ ở điều kiện bình thường. Nó mang chính `eventId` nói trên và thực thi idempotent, nên bản relay từ outbox đến sau không tạo cập nhật trùng — kể cả khi lệnh cập nhật đồng bộ bị timeout nhưng thực chất đã thành công. Lưu ý `eventId` chỉ chặn trùng lặp của *cùng một event*, không chặn sai thứ tự giữa *các event khác nhau*: sự kiện phải mang thêm `scoreVersion` tăng dần, và Redis Sorted Set chỉ chấp nhận version mới hơn bản ghi hiện có (so sánh nguyên tử bằng Lua script) — nếu không, bản relay trễ của event cũ có thể ghi đè điểm mới.

Một biến thể cần cảnh giác là "thử sync trước, thất bại mới ghi outbox". Cách này **không** đóng được cửa sổ dual-write: nếu dịch vụ sập sau khi commit database mà chưa kịp ghi outbox, cập nhật mất vĩnh viễn.

Hình dưới vẽ đường CQRS bất đồng bộ qua Kafka — chính luồng sinh ra replication lag vừa nói; phương án Redis đồng bộ không có trong hình.



Hình 7.10: *Đường CQRS bất đồng bộ qua Kafka cho Leaderboard — luồng này sinh replication lag; phương án đồng bộ Redis Sorted Set được mô tả trong văn bản*

CQRS ≠ Event Sourcing

CQRS và Event Sourcing thường được nhắc đến cùng nhau, nhưng chúng là **hai pattern độc lập**. CQRS tách read/write model — có thể dùng mà không cần Event Sourcing. Event Sourcing lưu event thay vì state — có thể dùng mà không cần CQRS. Kết hợp cả hai rất mạnh nhưng cũng *rất phức tạp*. Richardson trong [2a, Ch.6] khuyến nghị: bắt đầu với CQRS đơn giản trước, thêm Event Sourcing *khi có nhu cầu cụ thể* (audit trail, temporal query).

7.5. Event Sourcing — lưu event thay vì state

Nếu một bảng điểm chỉ ghi nhận kết quả cuối cùng mà không lưu lại quá trình học tập, thi lại hay phúc khảo, hệ thống đã đánh mất phần lịch sử quan trọng nhất. Event Sourcing giữ lại đúng phần đó: nó lưu chuỗi biến động thay vì trạng thái sau cùng, giống một cuốn sổ học vụ ghi đủ mọi lần thay đổi.

7.5.1. Vấn đề: mất lịch sử khi chỉ lưu state

Với database truyền thống, chúng ta lưu **trạng thái hiện tại** (*current state*): `submission.status = JUDGED, score = 85`. Mọi thay đổi trước đó bị ghi đè — không biết submission đã qua những trạng thái nào, ai thay đổi, khi nào.

Trong nhiều domain (tài chính, audit, legal), lịch sử thay đổi có giá trị ngang bằng hoặc hơn trạng thái hiện tại. Ngay cả trong KBM: “sinh viên nộp bài 3 lần, lần đầu sai, lần 2 đúng một phần, lần 3 đúng hoàn toàn” — thông tin này giá trị cho phân tích học tập, nhưng nếu chỉ lưu `status = CORRECT`, lịch sử bị mất.

7.5.2. Event Sourcing pattern

Event Sourcing thay đổi cách lưu trữ: thay vì lưu state, lưu **chuỗi sự kiện** đã xảy ra. State hiện tại được **derive** bằng cách replay tất cả event [2a, Ch.6]:



Hình 7.11: *Event Sourcing — lưu chuỗi sự kiện thay vì chỉ lưu trạng thái hiện tại*

Kleppmann trong [7, Ch.11] giải thích: Event Sourcing coi event log là **nguồn sự thật** (*source of truth*), còn các khung nhìn trạng thái (bảng cơ sở dữ liệu, materialized view) là **derived data** — có thể dựng lại bất kỳ lúc nào bằng cách phát lại (replay) event.

Lịch sử biến động và bảng điểm tổng kết

Các kỹ sư phần mềm đã quen thuộc với mô hình CRUD và câu lệnh `UPDATE` thường gặp khó khăn khi làm quen với Event Sourcing. Hãy nghĩ đơn giản:

- **Event Store**: giống như **Lịch sử biến động điểm/Nhật ký chấm bài** (ghi nhận mọi giao dịch cộng điểm, trừ điểm). Không thể dùng bút xóa sửa lại quá khứ, chỉ có thể ghi thêm sự kiện mới. Nhưng việc lấy nhật ký chấm bài ra để cộng lại xem hiện tại sinh viên đang có bao nhiêu điểm là rất chậm. Hơn nữa, truy vấn kiểu `SELECT * FROM nhat_ky WHERE so_diem_hien_tai > 10` không thực hiện được ngay trên nhật ký — `so_diem_hien_tai` không nằm trong bất kỳ dòng nhật ký nào mà là kết quả *cộng dồn* của cả chuỗi sự kiện (điều này đúng kể cả khi event store được cài trên chính một relational database).
- **Projection (Materialized View)**: giống như **Bảng điểm tổng kết**. Hệ thống tổng hợp toàn bộ dữ liệu sự kiện trong quá khứ, tính toán và xuất ra kết quả "Điểm hiện tại". Khi UI cần hiển thị, nó chỉ cần đọc bảng điểm này. Trong thực tế, projection gần như luôn cần thiết cho các truy vấn *xuyên aggregate* và cho UI; còn để dựng trạng thái của *một* thực thể, hệ thống chỉ cần replay stream sự kiện của chính thực thể đó.

Bảng 7.7 tóm tắt ưu và nhược điểm của Event Sourcing.

Bảng 7.7: Ưu và nhược điểm của Event Sourcing

Ưu điểm	Nhược điểm
Full audit trail — mọi thay đổi đều được ghi	**Complexity** — rebuild state cần replay, vấn đề về hiệu năng
Temporal query — "state tại thời điểm T?"	**Event schema evolution** — event cũ có format khác event mới
Hỗ trợ gỡ lỗi (debug) thuận tiện — có thể phát lại (replay) các sự kiện để tái hiện lỗi (reproduce bug)	**Eventual consistency** — khi projection cập nhật bất đồng bộ, read model "chậm hơn" event stream và cần xử lý khoảng trễ
Replay — rebuild state, tạo view mới	**Storage** — event log tăng theo thời gian, cần retention/ archiving phù hợp (snapshot chỉ giải bài toán tốc độ replay)
Kết hợp CQRS — event tự nhiên feed vào read model	**Learning curve** — đòi hỏi sự thay đổi tư duy hoàn toàn so với CRUD

Tuy nhiên, hệ thống Event Sourcing không thể phát lại (replay) toàn bộ sự kiện mỗi khi cần truy vấn trạng thái hiện tại. Điều này tương tự như việc phòng giáo vụ không thể lật lại từng bài kiểm tra từ năm nhất chỉ để tính điểm tích lũy. Giải pháp ở đây là tạo ra các **snapshot** — bản chụp trạng thái tại một mốc sự kiện.

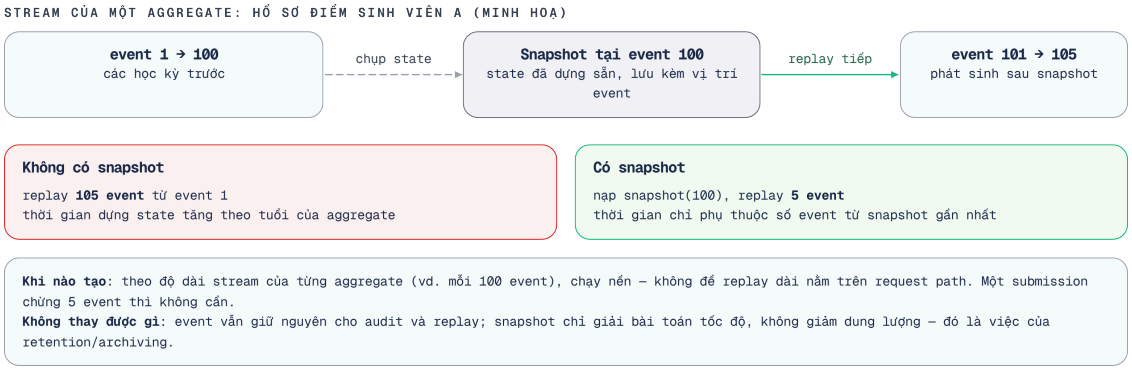
⚠ Snapshot trong Production

Tiêu chí quyết định snapshot là **độ dài stream của từng aggregate**, không phải tổng số event toàn hệ thống. Một submission chỉ có 5 event thì không bao giờ cần snapshot; nhưng một aggregate sống lâu (ví dụ hồ sơ điểm tích lũy của một sinh viên qua nhiều học kỳ) sẽ khiến mỗi lần dựng lại trạng thái chậm dần theo thời gian. Hãy quyết định dựa trên SLO rehydration đo được — và lưu ý snapshot không giải quyết bài toán dung lượng lưu trữ; đó là việc của retention/ archiving.

Điều cần tránh tuyệt đối là để việc replay không giới hạn nằm trên *request path*: chờ người dùng gọi API GET mới bắt đầu replay hàng trăm nghìn event sẽ làm độ trễ tăng vọt. Ngoài nguyên tắc đó, count-based (mỗi N event của một aggregate), time-based (theo lịch, ngoài giờ cao điểm — cách LMAX từng dùng) hay on-demand có kiểm soát đều là lựa chọn hợp lệ — chọn theo độ dài stream, tốc độ ghi và SLO phục hồi. Với KBM, count-based chạy nền theo từng aggregate là điểm khởi đầu đơn giản và an toàn nhất.

7.5.3. Snapshot Pattern — giải quyết hiệu năng của event replay

Khi stream của *một aggregate* dài ra (ví dụ hồ sơ điểm của một sinh viên tích lũy hàng nghìn event sau nhiều học kỳ), thời gian dựng lại trạng thái bằng cách replay từ event đầu tiên sẽ **tăng dần theo tuổi đời của aggregate**. **Snapshot pattern** giải quyết:



Hình 7.12: Snapshot pattern — replay từ snapshot thay vì từ đầu

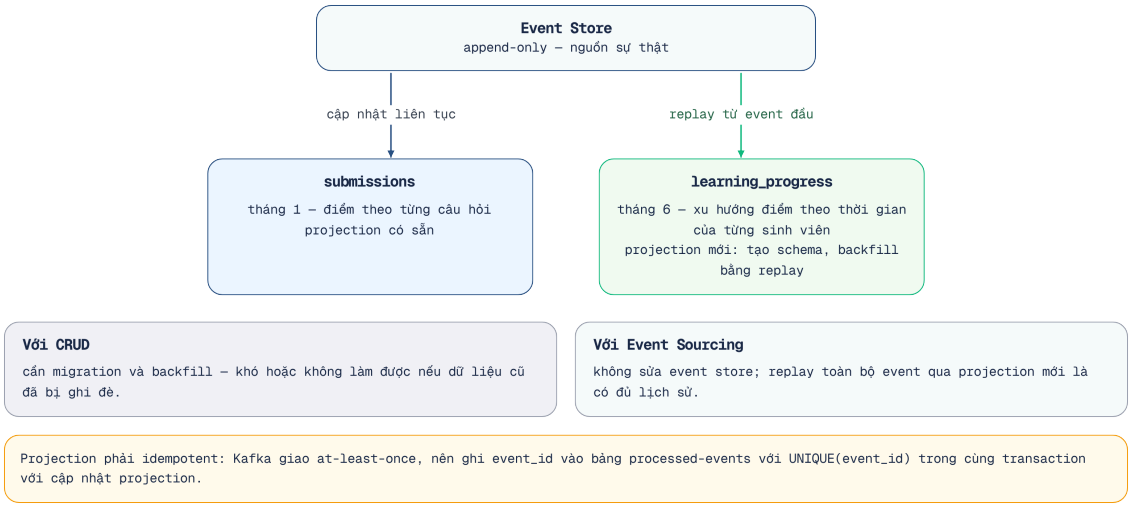
Chiến lược tạo snapshot trong Event Sourcing được thực hiện ở cấp độ **Từng Aggregate (Thực thể)**. Do đó, chiến lược phổ biến là dựa trên số lượng (**Count-based**): ví dụ cứ sau 100 event của một thực thể thì tạo một bản snapshot mới.

7.5.4. Projection Rebuilds — tạo lại view từ event

Một trong những lợi ích mạnh nhất của Event Sourcing: **tạo view mới mà không phải sửa event store hay suy diễn lại lịch sử từ current state** (read model mới vẫn cần được tạo schema rồi backfill bằng replay). Ví dụ giả định với KBM:

1. Tháng 1: chỉ cần bảng `submissions` (điểm theo câu hỏi)
2. Tháng 6: cần thêm bảng `learning_progress` (xu hướng điểm theo thời gian của từng sinh viên)
3. Với CRUD: cần database migration + backfill dữ liệu (khó hoặc không thể nếu dữ liệu cũ đã bị ghi đè)
4. Với Event Sourcing: Hệ thống chỉ cần phát lại (replay) toàn bộ sự kiện qua một projection mới, từ đó bảng `learning_progress` sẽ tự động được dựng lại cùng với toàn bộ dữ liệu lịch sử (full history)

Hình 7.13 minh họa kịch bản trên: projection mới được dựng lại bằng cách phát lại event store.



Hình 7.13: *Projection Rebuilds — tạo view mới từ event mà không cần migration schema*

Khi CQRS tiêu thụ event để dựng Read Model (Projection), cần đặc biệt chú ý đến **idempotency**: message broker (như Kafka) thường chỉ đảm bảo *at-least-once delivery*, nghĩa là một event có thể bị gửi lặp lại. Do đó, hàm cập nhật Projection bắt buộc phải có tính idempotent. Cách làm chắc chắn là một bảng inbox/processed-events với ràng buộc `UNIQUE(event_id)`, được ghi *trong cùng transaction* với cập nhật projection — event trùng sẽ vi phạm UNIQUE và bị bỏ qua, tránh cộng dồn sai lệch. Nếu cần chống cả event *sai thứ tự*, so sánh thêm version/sequence của aggregate; đừng dựa vào một mốc `last_processed_event_id` duy nhất, vì `eventId` là định danh duy nhất nhưng không mang thứ tự.

Bảng 7.8: *Event Store — các công cụ phổ biến*

Event Store	Mô tả	Phù hợp khi
EventStoreDB	Database chuyên dụng cho Event Sourcing (by Greg Young)	Cần đủ tính năng ES: subscription, projection, temporal query
Axon Framework	Java framework cho CQRS + ES, tích hợp Spring	Nhóm Java/Spring, cần opinionated framework
Kafka (kèm lưu ý)	Message broker với retention vô hạn	Đã dùng Kafka, chỉ cần event streaming (không cần truy vấn theo aggregate)
PostgreSQL + custom	Relational DB với bảng sự kiện (event table)	Phù hợp với nhóm phát triển quy mô nhỏ, ưu tiên sự đơn giản và muốn sử dụng hệ quản trị cơ sở dữ liệu sẵn có

Dù có nhiều lựa chọn, Kafka vẫn thường bị nhầm lẫn với các kho lưu trữ sự kiện thuần túy.

Kafka với retention vĩnh viễn (infinite retention) có thể hoạt động như event store. KBM đã dùng Kafka cho submission pipeline (Ch.5). Về lý thuyết, các thông điệp trên topic `submissions` và `judge-results` *chính là* chuỗi event. Tuy nhiên, Kafka được thiết kế là message broker, không phải event store chuyên dụng — truy vấn event theo entity ID khó hơn so với EventStoreDB hoặc Axon. Kleppmann trong [7, Ch.11] phân tích: Kafka phù hợp cho **event streaming** (*process events real-time*), còn Event Sourcing cần **event store** (*query events by aggregate*). Hai bài toán liên quan nhưng khác nhau.

Kafka ≈ Event Store?

7.5.5. Tiến hóa lược đồ sự kiện (Event Schema Evolution)

Event, giống API (Ch.3), cần **schema evolution** khi nghiệp vụ thay đổi. Nhưng khác API: các event đã lưu *không thể sửa* — event store là append-only. Hai chiến lược:

1. Upcasting — Khi đọc các event cũ, chuyển đổi (transform) sang định dạng mới trước khi áp dụng:

Listing 7.1: *Upcasting — chuyển event cũ sang định dạng mới*

```
// Event v1: { type: "SubmissionCreated", userId: "123", sql: "SELECT ..." }
// Event v2: { type: "SubmissionCreated", userId: "123", queryStr: "SELECT ..." }
// Upcaster: xử lý Breaking Change: đổi tên field 'sql' thành 'queryStr'
```

2. Weak schema — Event chứa các trường bổ sung (extra field) mà consumer bỏ qua nếu không hiểu (Tolerant Reader — Ch.3). Thêm trường mới → các event cũ vẫn hợp lệ.

Event là Fact, không phải Command

Event mô tả *điều đã xảy ra* (past tense: `SubmissionJudged`), không phải *yêu cầu* (`JudgeSubmission`). Event có tính bất biến (immutable) — không bao giờ sửa/xóa event đã lưu. Nếu cần "undo", thêm event mới: `SubmissionResultCorrected`. Richardson trong [2a, Ch.6] nhấn mạnh: đây là *quyết định nghiệp vụ*, không phải *quyết định kỹ thuật* — chuyên gia nghiệp vụ nên đặt tên cho các event.

7.5.6. Khi nào dùng Event Sourcing?

Bảng 7.9 đánh giá mức phù hợp của Event Sourcing theo từng kịch bản.

Bảng 7.9: *Khi nào dùng Event Sourcing*

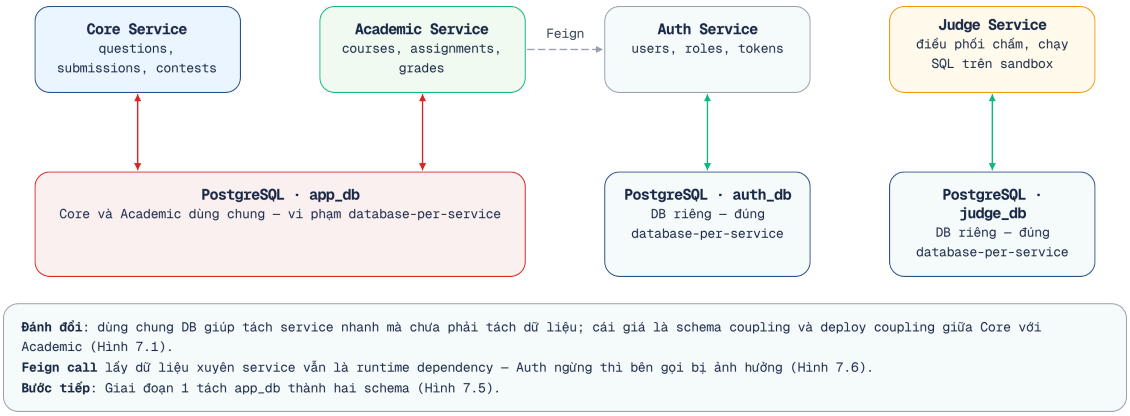
Kịch bản	Phù hợp?	Lý do
Yêu cầu kiểm toán (tài chính, compliance)	√ Rất phù hợp	Cần lịch sử đầy đủ, không thể xóa
Temporal analysis (phân tích xu hướng)	√ Phù hợp	Truy vấn "trạng thái tại thời điểm X" tự nhiên
CRUD đơn giản (quản lý user, cấu hình)	× Không	Chi phí vận hành và độ phức tạp phát sinh quá lớn so với lợi ích mang lại
Domain phức tạp (đơn hàng, booking)	√ Phù hợp	Sự kiện nghiệp vụ ánh xạ trực tiếp vào domain event
KBM submission tracking	! Tùy	Có giá trị cho phân tích học tập, nhưng hiện tại nhóm phát triển nhỏ → không ưu tiên

7.6. Case study: KBM

Sau khi đã đi qua các nguyên tắc quản lý dữ liệu phân tán, phần này áp dụng chúng vào KBM: nhận diện kiến trúc dữ liệu hiện tại, chỉ ra khoảng trống so với giải pháp khuyến nghị, và đề xuất lộ trình chuyển đổi theo giai đoạn.

7.6.1. Hiện trạng

Phân tích source code KBM cho thấy kiến trúc dữ liệu hiện tại:



Hình 7.14: Kiến trúc dữ liệu hiện tại của KBM — shared database là thiếu sót chính

Quan sát chính:

1. **Auth Service** — đã tách database riêng (`auth_db`) → ✓ đúng database-per-service
2. **Core + Academic** — chia sẻ `app_db` → ! nợ kỹ thuật chuyển tiếp có điều kiện (cần xác lập ranh giới sở hữu bảng trước khi tách)
3. **Judge Service** — database riêng (`judge_db`: dữ liệu test case, bảng đã tạo trong sandbox, bài nộp phía judge) → ✓ đúng database-per-service
4. **Cross-service data** — dùng Feign call → ! runtime dependency

Phân tích query pattern: KBM sử dụng hai pattern khi truy vấn dữ liệu vượt ra ngoài một entity:

- **Interface Projections** (Spring Data JPA) — tối ưu truy vấn *nội bộ* (kể cả cross-schema trên shared database hiện tại): trả về lightweight projection thay vì full entity, giảm lượng dữ liệu truyền tải. Đây không phải cơ chế vượt ranh giới dịch vụ.
- **Feign call** — pattern cross-service thực sự: gọi API của service khác khi cần dữ liệu ngoài ranh giới.

7.6.2. Từ hiện trạng đến kiến trúc mục tiêu

Bảng 7.10 tổng hợp các vấn đề dữ liệu của KBM và chiến lược chuyển đổi cho từng vấn đề.

Bảng 7.10: Tổng hợp vấn đề quản lý dữ liệu trong KBM và chiến lược chuyển đổi

#	Vấn đề	Hiện trạng KBM	Chiến lược chuyển đổi
1	**Shared database**	Core + Academic cùng `app_db`	Separate schema + phân quyền role (tách vật lý khi cần mở rộng)
2	**Cross-service join**	Cross-table query trực tiếp	Thay bằng Feign hoặc bản sao qua event tùy luồng
3	**Leaderboard query**	Truy vấn tổng hợp trên `app_db` kết hợp lấy thông tin người dùng qua API	CQRS — pre-computed read model

#	Vấn đề	Hiện trạng KBM	Chiến lược chuyển đổi
4	**Submission history**	CRUD (chỉ lưu current state)	(Tuỳ chọn, chưa ưu tiên) Event Sourcing hoặc bảng lịch sử trạng thái cho phân tích
5	**User reference data**	Feign call mỗi khi cần	Bản sao cục bộ qua event
6	**Grade Scheme / Attendance / MCQ Daily / CLO**	Nhiều read model học vụ cần tổng hợp dữ liệu khóa học, lớp, bài làm, chuẩn đầu ra	Xác định owner rõ ràng, nhân bản chỉ-đọc qua event hoặc materialized view
7	**Schema migration không đồng nhất**	Các service Java thiên về JPA auto-DDL; các service Go trong DevOps Lab phù hợp hơn với golang-migrate	Chuẩn hóa migration bằng versioned script, không dựa vào auto-DDL ở production

7.6.3. Lộ trình chuyển đổi đề xuất

Lộ trình dưới đây mở rộng 3 giai đoạn tách database ở Bảng 7.4 thành kế hoạch quản lý dữ liệu đầy đủ: hai giai đoạn đầu dựng ranh giới schema và chọn đường truy cập dữ liệu theo bài toán; Full Split (giai đoạn 3 của bảng) không nằm trong lộ trình ngắn hạn và chỉ thực hiện khi phát sinh nhu cầu mở rộng độc lập; các giai đoạn 3–5 bổ sung phần nhân bản dữ liệu tham chiếu, bảng xếp hạng và read model học vụ.

Giai đoạn 1 — Kiểm kê sở hữu và tách schema (ưu tiên cao, chi phí thực hiện thấp): Kiểm kê toàn bộ bảng trong `app_db`, khóa một chủ sở hữu ghi duy nhất cho từng bảng và tắt `JPA auto-DDL` cạnh tranh (chỉ một pipeline migration quản lý schema của mỗi dịch vụ). Sau đó tách `app_db` thành schema `core_schema` và `academic_schema`, mỗi service một database role riêng. Truy vấn chéo còn lại chỉ được đọc qua các view được cấp quyền rõ ràng (`GRANT SELECT` trên view, không trên bảng) — danh sách view này chính là danh sách việc của bước kế tiếp. Chưa thu hồi quyền đọc chéo ở bước này để không làm hỏng chức năng đang chạy.

Giai đoạn 2 — Truy cập dữ liệu qua API cho luồng cần dữ liệu tức thời (Chi phí thực hiện trung bình): Với các thao tác ghi hoặc luồng đọc ít nhưng đòi hỏi dữ liệu mới tức thời (ví dụ kiểm tra điều kiện nộp bài, trạng thái bài thi), thay truy vấn xuyên schema bằng Feign call — Academic gọi API của Core (như `GET /api/questions/batch`) thay vì đọc trực tiếp `core_schema.questions`. Khi các view đọc chéo lần lượt được thay bằng API (hoặc bản sao sự kiện ở Giai đoạn 3) và không còn ai dùng, gỡ view và chặn cứng ranh giới bằng `REVOKE ALL ON SCHEMA core_schema FROM academic_role;`.

Giai đoạn 3 — Nhân bản dữ liệu tham chiếu qua sự kiện (Chi phí thực hiện trung bình): Thay vì gọi API đồng bộ lặp đi lặp lại trên mỗi lần tải trang, với dữ liệu tham chiếu đọc nhiều và ít thay đổi, Core/Auth phát sự kiện (`QuestionUpdated`, `UserUpdated`) khi dữ liệu gốc đổi; Academic tiêu thụ và lưu bản sao cục bộ chỉ-đọc (tiêu đề, độ khó câu hỏi, họ tên người dùng) để giảm phụ thuộc thời gian chạy.

Lưu ý về khởi tạo dữ liệu ban đầu (bootstrapping): Khi áp dụng Data Duplication qua sự kiện, cần thiết kế thêm cơ chế "Bulk Export/Sync" hoặc "Reconciliation" (đối soát định kỳ) để khởi tạo dữ liệu ban đầu khi triển khai hoặc dựng môi trường mới — nếu không, service tiêu thụ sẽ khởi động với bản sao rỗng và chỉ có dữ liệu từ thời điểm bật event trở đi. Việc phát sự kiện cũng phải đảm bảo tính nguyên tử (dùng Transactional Outbox Pattern ở Ch.10) để đề phòng lỗi mạng.

Giai đoạn 4 — Leaderboard thời gian thực với Redis (Chi phí thực hiện cao, giá trị lớn cho chế độ thi): Xây dựng bảng xếp hạng thành read model trong bộ nhớ bằng Redis Sorted Set ngay tại `kbm-core` (hoặc `kbm-exam` khi tách phân hệ thi), cập nhật đồng bộ với **Composite Score** trên đường chính kết hợp Transactional Outbox bền vững, đồng thời ghép họ tên sinh viên từ bản sao tham chiếu `UserUpdated`. Truy vấn `ZREVRANK` chạy in-memory với độ phức tạp `O(log N)` — đủ nhanh cho hiển thị thời gian thực. Ở điều kiện fast path thành công, người nộp bài đọc lại được ngay điểm vừa ghi (read-after-write trên primary); khi fast path lỗi, bản relay từ outbox vẫn cập nhật bất đồng bộ — tức độ trễ chỉ được loại bỏ trên đường chính, không phải "tuyệt đối".

Giai đoạn 5 — Read model cho phân tích học tập (khi mở rộng học vụ): Grade Scheme, Attendance, MCQ Daily và CLO nên được xem như các read model học vụ. Owner của dữ liệu gốc vẫn nằm trong bounded context tương ứng; bảng tổng hợp chỉ phục vụ truy vấn nhanh cho giảng viên và báo cáo, không trở thành nguồn sự thật thứ hai.

⚠ Sai lầm thường gặp khi quản lý dữ liệu phân tán

- **Chia database quá sớm**: Tách database trước khi hiểu rõ các mẫu truy cập dữ liệu (data access pattern). Hậu quả: phát hiện hai service cần JOIN dữ liệu liên tục, phải xây dựng luồng dữ liệu sự kiện (event pipeline) phức tạp cho một tác vụ mà trước đây chỉ cần một câu lệnh SQL JOIN đơn giản. *Phòng tránh*: Bắt đầu với separate schema (cùng DB server), theo dõi các truy vấn xuyên schema trong 2–4 tuần, rồi mới quyết định tách hoàn toàn.
- **Dùng CQRS cho mọi thứ**: Áp dụng CQRS/Event Sourcing cho các chức năng CRUD cơ bản. Hậu quả: độ phức tạp của hệ thống tăng vọt, đội ngũ phát triển lãng phí thời gian bảo trì luồng sự kiện cho những dữ liệu rất ít khi thay đổi. *Phòng tránh*: Chỉ áp dụng CQRS cho các luồng truy vấn phức tạp (yêu cầu nối dữ liệu liên dịch vụ, lưu lượng đọc lớn, cần giao diện theo thời gian thực). Kiến trúc CRUD cơ bản đã đáp ứng đủ cho phần lớn các bài toán thông thường.
- **Nhân bản dữ liệu nhưng không xác định source of truth**: Sao chép dữ liệu giữa các service mà không rõ "ai sở hữu dữ liệu này?". Hậu quả: hai service cùng sửa dữ liệu → xung đột, không biết bản nào đúng. *Phòng tránh*: Mỗi thực thể dữ liệu chỉ có **duy nhất một service sở hữu** — các bản sao khác là bản sao chỉ-đọc (read-only replica), chỉ cập nhật qua event từ service sở hữu.
- **Lạm dụng eventual consistency cho Real-time UX**: Áp dụng async CQRS (như qua Kafka) cho tính năng cần cập nhật gần như tức thời như Live Leaderboard. Hậu quả: người dùng nộp bài thành công nhưng bảng xếp hạng chưa cập nhật; Optimistic UI không thể áp dụng ở đây vì client không thể tự tính thứ hạng khi nó phụ thuộc vào điểm số của các người dùng khác. *Phòng tránh*: Với các bài toán yêu cầu cập nhật bảng xếp hạng theo thời gian thực (như thi đấu trực tuyến), hãy dùng cấu trúc dữ liệu trong bộ nhớ cập nhật đồng bộ trên primary (như **Redis Sorted Set** — cho read-after-write ngay trên đường ghi chính) thay vì async event replication.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

Xem minh họa động tại companion web:

- **Mô hình CQRS và Event Sourcing**: cqrs-event-sourcing.html
- **Event-Carried State Transfer (nhân bản dữ liệu qua event)**: event-carried-state-transfer.html

7.7. Tổng kết

Quản lý dữ liệu là một trong những bài toán phức tạp nhất khi chuyển từ monolith sang microservices — phức tạp hơn cả việc tách application code. Database-per-service không chỉ là nguyên tắc kỹ thuật mà là **điều kiện tiên quyết** cho khả năng triển khai độc lập — mục tiêu cốt lõi của microservices. CAP Theorem (cùng mở rộng PACELC) nhắc nhở: trong hệ thống phân tán, tính nhất quán, tính sẵn sàng và độ trễ luôn đòi hỏi sự đánh đổi có chủ đích theo từng luồng dữ liệu.

Năm chiến lược tách database (view → wrapping service → separate schema → data transfer → full split) cho phép chuyển đổi **tăng dần** — không cần "big bang". Với nhóm phát triển nhỏ như KBM, kiểm kê chủ sở hữu bảng và bắt đầu từ separate schema là bước đi thực tế nhất.

Data duplication không phải anti-pattern — đây là **trade-off có chủ đích** để giảm runtime coupling. Nguyên tắc: mỗi tập dữ liệu chỉ có một source of truth, các bản sao là bản sao chỉ-đọc (read-only replica) được cập nhật qua event. Newman đã nói rõ: "Duplication is far better than coupling."

CQRS (mở rộng từ nguyên tắc CQS của Meyer) tách read model và write model — giải quyết bài toán truy vấn xuyên service (cross-service query) và lưu lượng đọc lớn (high-volume read). Event Sourcing bổ sung bằng cách lưu lịch sử đầy đủ thay vì chỉ trạng thái hiện tại. Cả hai pattern này mang lại nhiều lợi ích nhưng rất phức tạp — chỉ áp dụng khi có nhu cầu cụ thể, không phải mặc định.

Một bài toán liên quan chặt chẽ là **giao dịch phân tán (distributed transaction)** — khi một nghiệp vụ cần thay đổi dữ liệu ở nhiều service. Saga pattern (đã thảo luận chi tiết ở Ch.6) là giải pháp: chuỗi local transaction + compensating action thay vì distributed ACID transaction. Saga và quản lý dữ liệu bổ trợ nhau: database-per-service tạo ra nhu cầu Saga; còn Saga có thể dùng chung hạ tầng domain-event/outbox với các projection CQRS hay hệ Event Sourcing — nhưng không phụ thuộc vào hai pattern đó.

Phân tích KBM cho thấy nợ kỹ thuật lớn nhất ở tầng dữ liệu là shared database giữa Core và Academic. Lộ trình chuyển đổi đi theo từng cổng kiểm chứng: khóa sở hữu & tách schema → chọn API hoặc bản sao sự kiện tùy bài toán → CQRS với Redis Sorted Set cho bảng xếp hạng trực tiếp và read model cho phân tích học tập. Mỗi giai đoạn độc lập, có thể dừng ở bất kỳ điểm nào khi đã đáp ứng đủ cho ngữ cảnh đội ngũ.

Ở Chương 8, chúng ta sẽ chuyển sang **API Gateway** — điểm vào tập trung cho hệ thống microservices: định tuyến (routing), xác thực, giới hạn lưu lượng (rate limiting), và cách KBM sử dụng Spring Cloud Gateway.

7.8. Đọc thêm

Sách tham khảo chính:

1. [4b] Sam Newman, *Monolith to Microservices* — Ch.4: Decomposing the Database, Database-per-service patterns
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.6: Event Sourcing; Ch.7: Implementing Queries (API Composition, CQRS)
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.5: Replication; Ch.6: Partitioning; Ch.7,9: Transactions & CAP; Ch.11: Stream Processing, Event Sourcing

Sách bổ trợ:

4. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.5: Data Delegate Pattern, Event Sourcing in practice
5. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — §4.5: CQS vs CQRS, Command Sourcing; Ch.5: CAP in real world, eventual consistency
6. [6] Eric Evans, *Domain-Driven Design* — Aggregate design → data ownership per bounded context

Chương liên quan:

- **Ch.6 — Saga Pattern**: Giao dịch phân tán, orchestration vs choreography, compensating action. Saga giải quyết bài toán "thay đổi dữ liệu ở nhiều service trong một giao dịch nghiệp vụ" — bổ trợ trực tiếp cho database-per-service.

Nguồn trực tuyến:

- Martin Fowler, "CQRS" — martinfowler.com/bliki/CQRS.html
- Greg Young, "CQRS Documents" — cqrs.files.wordpress.com
- Confluent, "Event Sourcing with Kafka" — developer.confluent.io

Bài tập: Xem phần Bài tập & case study ở cuối sách để luyện tập:

- **7-0**: Ôn tập: các pattern quản lý dữ liệu ([L1])
- **7-1**: CAP Theorem trong thực tế — "Pick Two" có đúng không? (Trade-off Analysis [L3])
- **7-2**: Design: Chiến lược chuyển đổi database cho KBM (System Design [L2])

Tài liệu tham khảo

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[3] Mitra, R., & Nadareishvili, I. (2020). *Microservices: Up and Running*. O'Reilly Media.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[4b] Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.

[5] Rocha, H. (2022). *Practical Event-Driven Microservices Architecture*. Apress.

[6] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

[7] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.